**Question**

What is **breadth-first search** (BFS)? Use the pond analogy: in what order does it visit the cells, and what does it use to remember cells waiting their turn?



**Question**

A queue starts empty: push 4, push 7, pop, push 2, pop, pop. What does each pop return? Why does BFS use a queue and not a stack?



**Question**

List the steps of BFS in Project 2: the setup, what each pass of the loop does, and what happens after the loop.



**Question**

BFS marks a cell visited when it is **pushed**, not when it is popped. What goes wrong if you wait until the pop?



**Question**

BFS starts at S = (0,0), looks for T = (2,2), and checks neighbors left, right, up, down. Mark visited on push.

```
S . #
.   .   .
# .   T
```

List the cells in the order they are popped. How many cells are on the route?



**Question**

What does the **parent table** store, and why do we need it? What is the parent of the start cell?

**Answer**

BFS spreads outward in **layers**, like ripples from a stone dropped in a pond: first the start, then every cell 1 step away, then every cell 2 steps away, and so on.

A **queue** holds the discovered cells waiting to be explored, so nearer cells always come out before farther ones.



**Answer**

The pops return **4, 7, 2**: first in, first out.

A queue gives back the **oldest** discovered cell, so all cells at distance $k$ are handled before any at distance $k+1$. A stack gives back the newest cell and dives deep along one branch, which is DFS.



**Answer**

**Setup:** visited table, parent table (all `{-1,-1}`), queue; mark the start visited and push it.

**Loop** while the queue is not empty: pop `curr`; for each open, unvisited neighbor (left, right, up, down): mark it visited, set its parent to `curr`, stop if it is the end, else push it.

**After:** end not reached ⇒ empty stack; otherwise rebuild the path from `parent`.



**Answer**

Two cells at the same distance can both discover the same neighbor before it is popped. The neighbor can be pushed **more than once**, causing duplicate work. If its parent is assigned each time it is pushed, the parent can also be overwritten.

Marking on push means each cell is pushed exactly once.

Example: in an open 2×2 grid, `(1,1)` is pushed 2 times if marked on pop, 1 time if marked on push.



**Answer**

Pop order: `(0,0)`, `(0,1)`, `(1,0)`, `(1,1)`, `(1,2)`. The pop of `(1,2)` discovers T, so BFS stops (5 pops).

Route: `(0,0)`, `(0,1)`, `(1,1)`, `(1,2)`, `(2,2)`: **5 cells**.

Queue after each pop: [(0,1),(1,0)], [(1,0),(1,1)], [(1,1)], [(1,2),(2,1)].



**Answer**

`parent[r][c]` is the cell we were processing when we first discovered `(r,c)`.

The queue forgets how a cell was reached, so after BFS reaches the end we follow parent links from the end back to the start.

The start was never discovered by anyone, so its parent stays `{-1,-1}`.

**Question**

BFS found the end. Write the loop that fills `pathStack` with the **start on top**, using `parent`.



**Question**

Why is the first route BFS finds a **shortest** one (fewest cells)?



**Question**

On an empty 3×3 grid, S = (0,0), T = (2,2), both searches check neighbors left, right, up, down. How many cells are on the route DFS returns? On the route BFS returns?



**Question**

A maze has $n$ cells. What are the time and space complexity of BFS? Why is the work per cell constant?



**Question**

A maze is one winding corridor of 100,000 cells. Which uses more memory at one moment: recursive DFS or BFS? What is the risk?



**Question**

```
Maze_Solver* s = new BFS_Solver(maze, a, b);
auto p = s->getPath(); delete s;
```

Which `getPath()` runs, and why? Why is `delete s` safe?

**Answer**

Cells leave the queue in order of distance: every cell at distance $k$ before any cell at distance $k+1$.

So the **first** time the end is discovered, it is reached by a shortest route; no shorter route can exist, because it would have been discovered in an earlier layer. The end is already marked visited, so a later, longer route never replaces its parent.



**Answer**

```
pathStack.push(end);
Coord p = parent[end.row][end.col];
while (p.row != -1 && p.col != -1) {
  pathStack.push(p);
  p = parent[p.row][p.col];
}
```

The start is pushed **last**, so a stack (last in, first out) gives it back first. No reversing is needed.



**Answer**

**Time $O(n)$:** each cell is marked visited once, so it is pushed and popped at most once, and each pop looks at only 4 neighbors (about $4n$ steps in all).

**Space $O(n)$:** the visited table, the parent table and the queue.

DFS is also $O(n)$ time and $O(n)$ space.



**Answer**

**DFS: 9 cells.** It goes right to (0,2), down to (1,2), then left (1,1), (1,0), down (2,0), right (2,1), (2,2): the first route it finds.

**BFS: 5 cells**, e.g. (0,0), (0,1), (0,2), (1,2), (2,2).

DFS finds **a** route; BFS finds a **shortest** route.



**Answer**

`BFS_Solver::getPath` runs. `getPath` is **virtual** (pure virtual in `Maze_Solver`), so C++ picks the version at **run time** from the real object, not the pointer type. Point `s` at a `DFS_Solver` and the same line runs DFS.

`delete s` is safe because `~Maze_Solver` is **virtual**: the whole BFS_Solver is destroyed.



**Answer**

**Recursive DFS:** about 100,000 nested calls, so the call stack can overflow.

**BFS:** only one waiting cell in the queue at a time.

But in a wide, branching maze BFS must remember a whole layer of cells at once. Both are $O(n)$ space in the worst case.